

- 2. Define target users** - Identify who experiences the problem and who will use or buy the solution.
- 3. Write the user journey** - Describe the path from discovery to completing the main goal.
- 4. Define the MVP** - Choose the smallest version that delivers meaningful value.
- 5. Write requirements** - Record features, roles, pages, data, integrations, constraints, and acceptance criteria.
- 6. Create wireframes** - Sketch screens and information structure before visual design.
- 7. Design the interface** - Define components, navigation, typography, colors, accessibility, and responsive behavior.
- 8. Choose the technology stack** - Select frontend, backend, database, authentication, hosting, and supporting services.
- 9. Plan the architecture** - Define system boundaries, data flows, APIs, trust boundaries, and deployment model.
- 10. Create repositories** - Set up source control, README files, branches, ignore rules, and contribution practices.
- 11. Set up environments** - Prepare local development, testing, staging, and production.
- 12. Design the database** - Define entities, relationships, constraints, migrations, retention, and backups.
- 13. Plan authentication and authorization** - Define registration, login, recovery, sessions, roles, and permissions.
- 14. Design APIs** - Specify endpoints, data contracts, validation, errors, versioning, and access control.
- 15. Build the frontend** - Implement responsive screens, components, forms, state, and interactions.
- 16. Build the backend** - Implement business rules, services, APIs, processing, and integrations.
- 17. Connect the database** - Implement reliable data creation, reading, updating, deletion, and transactions.
- 18. Add access control** - Protect private resources and enforce authorization on the server.
- 19. Connect external services** - Integrate email, payments, storage, analytics, maps, or notifications as needed.
- 20. Test the software** - Run unit, integration, end-to-end, usability, accessibility, security, and performance tests.
- 21. Fix bugs and improve usability** - Resolve defects and reduce friction in important user journeys.
- 22. Set up security controls** - Manage secrets, validate inputs, use HTTPS, restrict permissions, log events, and update dependencies.
- 23. Set up CI/CD** - Automatically verify, build, and deploy approved changes.
- 24. Deploy staging** - Publish a production-like environment for realistic review.
- 25. Complete acceptance testing** - Confirm the product meets agreed user and business needs.
- 26. Deploy production** - Release the live system with production configuration and controlled access.
- 27. Add analytics and monitoring** - Measure behavior, conversion, errors, performance, availability, and cost.
- 28. Prepare documentation** - Document users, operations, architecture, APIs, deployment, security, and recovery.

- 29. Launch to users** - Release gradually, communicate clearly, and collect structured feedback.
- 30. Maintain and improve** - Operate, support, patch, learn, prioritize, and release the next versions.

4. The roadmap is iterative, not a waterfall

The numbered list provides order and completeness, but real teams revisit earlier decisions. Testing may expose a requirement problem. Security review may change architecture. Analytics may reveal that the customer journey is wrong. Product development is therefore a set of connected loops, not one irreversible line.

Discover Problem, users, evidence	Define Journey, MVP, requirements	Build Design, code, integrations	Verify Test, secure, accept	Operate Deploy, measure, improve
--	--	---	---------------------------------------	---

5. How Become SWE uses the roadmap

The roadmap gives us a map of the complete profession. The lessons document the actual route we take. We may visit a later roadmap topic because the live website needs it now, then return to a foundation topic to understand it more deeply.

Completed examples: Lesson 1 dealt with hosting, DNS, and deployment. Lesson 2 dealt with a lead-collection integration. Lesson 3 dealt with architecture and security boundaries. Lesson 4 dealt with product planning and customer journey. This order is not the standard product-development order; it is the order in which the real project produced valuable learning.

Rule for future lessons: each lesson should identify the related roadmap stage, but lesson numbers should continue to reflect the chronology of our learning journey.

6. Practical method for a new product

For a new commercial product, begin with steps 1-5 before selecting technology. Build the smallest end-to-end path through design, frontend, backend, data, and authentication. Test that path early. Add automation, monitoring, and documentation before scale makes weaknesses expensive.

Keep the product roadmap, technical backlog, and learning record separate. The roadmap explains where the product is going. The backlog explains what the team will do next. The learning record explains what was discovered and how capability changed.

7. Key lessons

A complete software product requires more than coding. Product thinking, design, architecture, data, security, testing, deployment, measurement, documentation, and maintenance are all part of software engineering.

A learning project should not pretend to be a perfectly ordered product project. Its value comes from making uncertainty, mistakes, decisions, and improvement visible.

Use the product-development roadmap to avoid missing important work. Use the learning journey to understand why that work matters and how to perform it.

Overall sequence

Problem -> Users -> Journey -> MVP -> Requirements -> Design -> Architecture -> Development -> Testing -> Security -> Deployment -> Monitoring -> Improvement

Become SWE | Ticker Bell Kft. | swe.tickerbell.biz